

一人多电话处理方案



2026年03月14日



v1.0



一人多电话处理方案

文档版本：v1.0

编写日期：2026 年 3 月 14 日

开发负责人：王舟

一、问题分析

1.1 典型场景

场景 1：一人多电话（同一人）

- 张三（手机：138****1234，工作：010-****5678）
 - 张三（手机：139****9876，家庭：028-****1111）
- 四个人物记录，实际是同一人

场景 2：多人共用一电话（不同人）

- 张三（电话：010-****8888，公司前台）
 - 李四（电话：010-****8888，公司前台）
 - 王五（电话：010-****8888，公司前台）
- 同一公司总机，实际是不同人

场景 3：电话变更（同一人）

- 张三（旧电话：138****1234，2020 年）
 - 张三（新电话：139****5678，2024 年）
- 换号，实际是同一人

场景 4：电话类型不同

- 张三（手机：138****1234）
 - 张三（微信：wxid_xxxx）
 - 张三（邮箱：zhangsan@email.com）
- 多种联系方式，实际是同一人

1.2 核心挑战

挑战	说明	解决思路
一人多电话	同一个人有多个联系方式	人物 - 电话一对多关联
多人一电话	公司总机/家庭电话共用	电话类型标记 + 上下文判断
电话变更	换号/停机/新增	时间戳记录 + 有效性标记
查重逻辑	如何避免误判	综合多字段判断，不单靠电话

二、数据库设计

2.1 人物 - 电话关联表（一对多）

```
-- 联系方式表（支持多种类型）
CREATE TABLE contact_methods (
  id          TEXT PRIMARY KEY,
  person_id   TEXT NOT NULL REFERENCES persons(id),

  -- 联系方式类型
  contact_type TEXT NOT NULL,      -- mobile/landline/work/home/email/wechat/etc.
  contact_value TEXT NOT NULL,     -- 联系方式值（加密存储）
  contact_hash TEXT NOT NULL,     -- 哈希值（用于快速匹配）

  -- 验证与状态
  is_primary   BOOLEAN DEFAULT FALSE, -- 是否主号码
  is_verified   BOOLEAN DEFAULT FALSE, -- 是否已验证
  verified_at   DATETIME,
  verified_by   TEXT,

  -- 时间信息
  first_seen_at DATETIME,          -- 首次出现时间
  last_used_at  DATETIME,          -- 最后使用时间
  invalid_at    DATETIME,          -- 失效时间（换号时标记）

  -- 来源与备注
  source        TEXT,              -- 来源: manual/evidence_extract/import
```

```

    note            TEXT,                -- 备注: 如"2020-2023 年使用"
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP,
    created_by      TEXT,

-- 索引
    INDEX idx_person ON contact_methods(person_id),
    INDEX idx_type_hash ON contact_methods(contact_type, contact_hash),
    INDEX idx_primary ON contact_methods(person_id, is_primary),
    UNIQUE INDEX idx_unique (contact_type, contact_hash)
);

-- 更新 persons 表, 移除独立 phone 字段或保留主号码 (冗余)
ALTER TABLE persons ADD COLUMN primary_phone TEXT;      -- 主号码 (冗余, 方便查询)
ALTER TABLE persons ADD COLUMN primary_phone_hash TEXT;

-- 电话类型字典表
CREATE TABLE contact_type_dict (
    id              TEXT PRIMARY KEY,
    code            TEXT UNIQUE NOT NULL,
    name            TEXT NOT NULL,
    category         TEXT NOT NULL,        -- phone/email/im
    icon             TEXT,                 -- 图标
    sort_order       INTEGER DEFAULT 0
);

-- 初始化联系方式类型
INSERT INTO contact_type_dict (id, code, name, category, icon) VALUES
('ct_001', 'mobile', '手机', 'phone', '📱'),
('ct_002', 'landline', '座机', 'phone', '☎️'),
('ct_003', 'work_phone', '工作电话', 'phone', '📞'),
('ct_004', 'home_phone', '家庭电话', 'phone', '🏠'),
('ct_010', 'email', '邮箱', 'email', '✉️'),
('ct_020', 'wechat', '微信', 'im', '💬'),
('ct_021', 'qq', 'QQ', 'im', '💬'),
('ct_022', 'sms', '短信', 'im', '📱'),
('ct_030', 'address', '地址', 'address', '📍');

```

2.2 人物查重逻辑调整

```

-- 人物相似度计算 (考虑一人多电话)
-- 不再简单依赖单一电话匹配

-- 视图: 人物联系方式汇总
CREATE VIEW person_contact_summary AS
SELECT

```

```

        p.id AS person_id,
        p.name,
        p.primary_phone,
        GROUP_CONCAT(DISTINCT cm.contact_type || ':' || cm.contact_value) AS all_contacts,
        COUNT(cm.id) AS contact_count
FROM persons p
LEFT JOIN contact_methods cm ON p.id = cm.person_id AND cm.invalid_at IS NULL
GROUP BY p.id;

-- 视图：电话被多人使用情况
CREATE VIEW phone_sharing_analysis AS
SELECT
    cm.contact_value,
    cm.contact_type,
    COUNT(DISTINCT cm.person_id) AS person_count,
    GROUP_CONCAT(DISTINCT p.name) AS person_names,
    GROUP_CONCAT(DISTINCT p.id) AS person_ids
FROM contact_methods cm
JOIN persons p ON cm.person_id = p.id
WHERE cm.contact_type IN ('mobile', 'landline', 'work_phone')
    AND cm.invalid_at IS NULL
GROUP BY cm.contact_value, cm.contact_type
HAVING person_count > 1;

```

2.3 人物合并记录扩展

```

-- 更新 person_merge_records 表，记录合并时的电话匹配情况
ALTER TABLE person_merge_records ADD COLUMN contact_match_details JSON;
/*
示例：
{
    "matched_contacts": [
        {"type": "mobile", "value": "138****1234", "status": "exact_match"}
    ],
    "new_contacts_added": [
        {"type": "work_phone", "value": "010-****5678", "source": "merged_person"}
    ],
    "conflicting_contacts": [
        {"type": "mobile", "kept": "138****1234", "discarded": "139****9878", "reason": "dup
    ]
}
*/

```

三、Rust 后端实现

3.1 联系方式数据模型

```
// src/models/contact.rs
use chrono::{DateTime, Utc, NaiveDateTime};
use serde::{Deserialize, Serialize};
use sqlx::FromRow;
use uuid::Uuid;

/// 联系方式类型
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq)]
#[serde(rename_all = "snake_case")]
pub enum ContactType {
    Mobile,          // 手机
    Landline,        // 座机
    WorkPhone,       // 工作电话
    HomePhone,       // 家庭电话
    Email,           // 邮箱
    Wechat,          // 微信
    QQ,              // QQ
    Address,         // 地址
}

/// 联系方式
#[derive(Debug, Clone, FromRow, Serialize, Deserialize)]
pub struct ContactMethod {
    pub id: String,
    pub person_id: String,
    pub contact_type: String,
    pub contact_value: String,
    pub contact_hash: String,
    pub is_primary: bool,
    pub is_verified: bool,
    pub verified_at: Option<DateTime<Utc>>,
    pub first_seen_at: Option<DateTime<Utc>>,
    pub last_used_at: Option<DateTime<Utc>>,
    pub invalid_at: Option<DateTime<Utc>>,
    pub source: Option<String>,
    pub note: Option<String>,
    pub created_at: DateTime<Utc>,
    pub created_by: String,
}
```

```

/// 人物联系方式汇总
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct PersonContactSummary {
    pub person_id: String,
    pub name: String,
    pub primary_phone: Option<String>,
    pub contacts: Vec<ContactMethod>,
    pub contact_count: i64,
}

impl ContactMethod {
    /// 创建新的联系方式
    pub fn new(
        person_id: String,
        contact_type: ContactType,
        contact_value: String,
        created_by: String,
    ) -> Self {
        Self {
            id: Uuid::new_v4().to_string(),
            person_id,
            contact_type: format!("{:?}", contact_type),
            contact_hash: Self::calculate_hash(&contact_type, &contact_value),
            contact_value,
            is_primary: false,
            is_verified: false,
            verified_at: None,
            first_seen_at: Some(Utc::now()),
            last_used_at: Some(Utc::now()),
            invalid_at: None,
            source: Some("manual".to_string()),
            note: None,
            created_at: Utc::now(),
            created_by,
        }
    }

    /// 计算哈希（用于去重）
    fn calculate_hash(contact_type: &ContactType, value: &str) -> String {
        use sha2::{Sha256, Digest};
        let mut hasher = Sha256::new();
        hasher.update(format!("{:?:{}}", contact_type, Self::normalize_value(value).as_byte
        format!("{:x}", hasher.finalize())
    }

    /// 标准化值（去除空格、横线等）

```

```

fn normalize_value(value: &str) -> String {
    value.chars()
        .filter(|c| c.is_alphanumeric() || c == &'+' || c == &'@' || c == &'.' )
        .collect::<String>()
        .to_lowercase()
}
}

```

3.2 联系方式服务

```

// src/services/contact_service.rs
use sqlx::{SqlitePool, FromRow};
use crate::models::contact::*;

pub struct ContactService {
    pool: SqlitePool,
}

impl ContactService {
    pub fn new(pool: SqlitePool) -> Self {
        Self { pool }
    }

    /// 添加联系方式（自动去重）
    pub async fn add_contact_method(
        &self,
        person_id: &str,
        contact_type: ContactType,
        contact_value: &str,
        created_by: &str,
    ) -> anyhow::Result<ContactMethod> {
        let contact_hash = ContactMethod::calculate_hash(&contact_type, contact_value);

        // 1. 检查是否已存在
        let existing = sqlx::query_as::<_, ContactMethod>(
            r#"
            SELECT * FROM contact_methods
            WHERE contact_hash = ? AND invalid_at IS NULL
            "#,
        )
        .bind(&contact_hash)
        .fetch_optional(&self.pool)
        .await?;

        if let Some(contact) = existing {

```

```

        // 已存在，检查是否属于同一人
        if contact.person_id == person_id {
            return Ok(contact); // 同一人，直接返回
        } else {
            // 属于不同人，记录冲突（用于后续判断）
            self.log_contact_conflict(&contact, person_id).await?;
        }
    }

    // 2. 创建新联系方式
    let mut contact = ContactMethod::new(
        person_id.to_string(),
        contact_type,
        contact_value.to_string(),
        created_by.to_string(),
    );

    // 3. 如果是第一个联系方式，设为主号码
    let contact_count = self.get_person_contact_count(person_id).await?;
    if contact_count == 0 {
        contact.is_primary = true;
    }

    // 4. 插入数据库
    self.insert_contact(&contact).await?;

    // 5. 更新 persons 表的主号码（冗余）
    if contact.is_primary && matches!(contact_type, ContactType::Mobile) {
        self.update_person_primary_phone(person_id, contact_value).await?;
    }

    Ok(contact)
}

/// 获取人物所有联系方式
pub async fn get_person_contacts(
    &self,
    person_id: &str,
) -> anyhow::Result<Vec<ContactMethod>> {
    let contacts = sqlx::query_as::<_, ContactMethod>(
        r#"
        SELECT * FROM contact_methods
        WHERE person_id = ?
        ORDER BY is_primary DESC, contact_type, created_at DESC
        "#,
    )
    .bind(person_id)

```



```

        .fetch_all(&self.pool)
        .await?;

        Ok(contacts)
    }

    /// 设置主号码
    pub async fn set_primary_contact(
        &self,
        person_id: &str,
        contact_id: &str,
    ) -> anyhow::Result<()> {
        let mut tx = self.pool.begin().await?;

        // 取消当前主号码
        sqlx::query(
            "UPDATE contact_methods SET is_primary = FALSE WHERE person_id = ? AND is_primary = TRUE"
        )
        .bind(person_id)
        .execute(&mut *tx)
        .await?;

        // 设置新主号码
        sqlx::query(
            "UPDATE contact_methods SET is_primary = TRUE WHERE id = ? AND person_id = ?",
        )
        .bind(contact_id)
        .bind(person_id)
        .execute(&mut *tx)
        .await?;

        tx.commit().await?;

        Ok(())
    }

    /// 标记电话失效（换号时）
    pub async fn mark_contact_invalid(
        &self,
        contact_id: &str,
        reason: &str,
    ) -> anyhow::Result<()> {
        sqlx::query(
            "UPDATE contact_methods SET invalid_at = ?, note = COALESCE(note, '') || ? WHERE id = ?"
        )
        .bind(Utc::now())
        .bind(format!("[失效: {}]", reason))

```

```

        .bind(contact_id)
        .execute(&self.pool)
        .await?;

    Ok(())
}

/// 通过联系方式查找人物（支持一个电话多人）
pub async fn find_persons_by_contact(
    &self,
    contact_type: ContactType,
    contact_value: &str,
) -> anyhow::Result<Vec<(Person, ContactMethod)>> {
    let contact_hash = ContactMethod::calculate_hash(&contact_type, contact_value);

    let results = sqlx::query_as::<_, (Person, ContactMethod)>(&
        r#"
        SELECT p.*, cm.*
        FROM persons p
        JOIN contact_methods cm ON p.id = cm.person_id
        WHERE cm.contact_hash = ? AND cm.invalid_at IS NULL
        "#,
    )
    .bind(&contact_hash)
    .fetch_all(&self.pool)
    .await?;

    Ok(results)
}

/// 获取共用电话的情况（用于提示）
pub async fn get_shared_contacts(
    &self,
    person_id: &str,
) -> anyhow::Result<Vec<SharedContactInfo>> {
    let shared = sqlx::query_as::<_, SharedContactInfo>(&
        r#"
        SELECT
            cm.contact_value,
            cm.contact_type,
            COUNT(DISTINCT cm.person_id) as person_count,
            GROUP_CONCAT(DISTINCT p.name) as person_names
        FROM contact_methods cm
        JOIN persons p ON cm.person_id = p.id
        WHERE cm.person_id != ? AND cm.invalid_at IS NULL
        AND cm.contact_value IN (

```

```

        SELECT contact_value FROM contact_methods WHERE person_id = ? AND invalid_
    )
    GROUP BY cm.contact_value, cm.contact_type
    HAVING person_count > 0
    "#,
)
.bind(person_id)
.bind(person_id)
.fetch_all(&self.pool)
.await?;

Ok(shared)
}

/// 查重时考虑一人多电话
pub async fn check_duplicate_with_multiple_contacts(
    &self,
    person_id: &str,
    new_contacts: &[ContactInput],
) -> anyhow::Result<Vec<DuplicateContactResult>> {
    let mut results = Vec::new();

    for contact in new_contacts {
        let persons = self.find_persons_by_contact(contact.r#type.clone(), &contact.valu

        for (existing_person, contact_method) in persons {
            let confidence = if existing_person.id == person_id {
                1.0 // 同一个人, 完全匹配
            } else {
                // 不同人但电话相同, 需要判断
                let shared_count = self.count_shared_contacts(person_id, &existing_perso
                if shared_count >= 2 {
                    0.8 // 两个以上电话相同, 大概率同一人
                } else {
                    0.5 // 仅一个电话相同, 可能是共用
                }
            }
        }

        results.push(DuplicateContactResult {
            existing_person_id: existing_person.id,
            contact: contact_method,
            confidence,
            is_same_person: existing_person.id == person_id,
        });
    }
}

```

```
        Ok(results)
    }
}

/// 共用电话信息
#[derive(Debug, Clone, FromRow, Serialize, Deserialize)]
pub struct SharedContactInfo {
    pub contact_value: String,
    pub contact_type: String,
    pub person_count: i64,
    pub person_names: String,
}

/// 联系方式输入
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct ContactInput {
    pub r#type: ContactType,
    pub value: String,
    pub is_primary: bool,
}

/// 查重结果
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct DuplicateContactResult {
    pub existing_person_id: String,
    pub contact: ContactMethod,
    pub confidence: f64,
    pub is_same_person: bool,
}
```

四、API 接口设计

4.1 联系方式管理

方法	路径	说明
GET	/api/v1/persons/:id/contacts	获取人物所有联系方式
POST	/api/v1/persons/:id/contacts	添加联系方式

方法	路径	说明
PUT	/api/v1/contacts/:id	更新联系方式
DELETE	/api/v1/contacts/:id	删除联系方式
POST	/api/v1/contacts/:id/set-primary	设为主号码
POST	/api/v1/contacts/:id/mark-invalid	标记失效

4.2 查重接口

```
POST /api/v1/persons/check-duplicate
```

请求体:

```
{
  "name": "张三",
  "contacts": [
    {"type": "mobile", "value": "138****1234"},
    {"type": "work_phone", "value": "010-****5678"},
    {"type": "email", "value": "zhangsan@company.com"}
  ],
  "organization": "XXX 公司"
}
```

响应:

```
{
  "code": 200,
  "data": {
    "duplicates": [
      {
        "person_id": "person_001",
        "name": "张三",
        "confidence": 0.95,
        "matched_contacts": [
          {"type": "mobile", "value": "138****1234"}
        ],
        "action": "suggest_merge"
      }
    ]
  }
}
```

```

    },
    {
      "person_id": "person_002",
      "name": "张三",
      "confidence": 0.5,
      "matched_contacts": [
        {"type": "work_phone", "value": "010-****5678"}
      ],
      "note": "该电话被 3 人共用, 可能是公司总机",
      "action": "manual_review"
    }
  ]
}

```

4.3 共用电话分析

```
GET /api/v1/persons/:id/shared-contacts
```

响应:

```

{
  "code": 200,
  "data": {
    "shared_contacts": [
      {
        "contact_value": "010-****8888",
        "contact_type": "landline",
        "person_count": 3,
        "person_names": ["张三", "李四", "王五"],
        "note": "可能是公司总机"
      }
    ]
  }
}

```

五、前端交互设计

5.1 添加联系方式组件

```
// frontend/src/components/person/ContactList.rs

#[component]
pub fn ContactList(person_id: String) -> Element {
    let contacts = use_resource(move || fetch_person_contacts(&person_id));
    let mut show_add_form = use_signal(|| false);

    rsx! {
        div { class: "contact-list",
            h4 { "联系方式" }

            match &*contacts.value() {
                Some(Ok(contacts)) => rsx! {
                    if contacts.is_empty() {
                        div { class: "empty", "暂无联系方式" }
                    }

                    for contact in contacts.iter() {
                        ContactItem {
                            contact: contact.clone(),
                            on_set_primary: move |_| set_primary_contact(&contact.id),
                            on_remove: move |_| remove_contact(&contact.id),
                            on_mark_invalid: move |_| mark_invalid(&contact.id),
                        }
                    }
                }

                _ => rsx! {}
            }

            button {
                class: "add-btn",
                onclick: move |_| show_add_form.set(true),
                "+ 添加联系方式"
            }

            if show_add_form() {
                AddContactForm {
                    person_id: person_id.clone(),
                    on_submit: move |_| {
                        show_add_form.set(false);
                        contacts.restart();
                    },
                    on_cancel: move |_| show_add_form.set(false),
                }
            }
        }
    }
}
```

```

        }
    },
    _ => rsx! { div { "加载中..." } }
}
}
}

#[component]
fn ContactItem(contact: ContactMethod) -> Element {
    let icon = get_contact_icon(&contact.contact_type);
    let type_name = get_contact_type_name(&contact.contact_type);

    rsx! {
        div { class: "contact-item {if contact.is_primary { "primary" } else { "" }}",
            span { class: "icon", {icon} }
            span { class: "type", {type_name} }
            span { class: "value", {contact.contact_value} }

            if contact.is_primary {
                span { class: "badge primary", "主号码" }
            }

            if contact.invalid_at.is_some() {
                span { class: "badge invalid", "已失效" }
            }

            div { class: "actions",
                if !contact.is_primary && contact.invalid_at.is_none() {
                    button {
                        class: "icon-btn",
                        onclick: move |_| on_set_primary.call(()),
                        title: "设为主号码",
                        "★"
                    }
                }

                if contact.invalid_at.is_none() {
                    button {
                        class: "icon-btn",
                        onclick: move |_| on_mark_invalid.call(()),
                        title: "标记失效",
                        "✕"
                    }
                }
            }
        }
    }
}

```



```

        button {
            class: "icon-btn",
            onclick: move |_| on_remove.call(()),
            title: "删除",
            "🗑️"
        }
    }
}
}
}
}

```

5.2 查重提示（考虑一人多电话）

```

// frontend/src/components/person/DuplicateCheckResult.rs

#[component]
pub fn DuplicateCheckResult(duplicates: Vec<DuplicateResult>) -> Element {
    rsx! {
        div { class: "duplicate-check-result",
            h4 { "⚠️ 发现相似人物" }

            for dup in duplicates.iter() {
                div {
                    class: "duplicate-item {dup.action}",

                    div { class: "header",
                        strong { "{dup.name}" }
                        span { class: "confidence", "相似度: {(dup.confidence * 100) as i32}%" }
                    }

                    // 匹配的联系方式
                    if !dup.matched_contacts.is_empty() {
                        div { class: "matched-contacts",
                            span { "匹配联系方式: " }
                            for contact in dup.matched_contacts.iter() {
                                span { class: "contact-tag",
                                    "{get_contact_icon(&contact.type)} {contact.value}"
                                }
                            }
                        }
                    }

                    // 共用电话提示
                    if dup.is_shared_contact {
                        div { class: "warning shared",

```



```

let mut score = 0.0;
let mut total_weight = 0.0;
let mut matched = Vec::new();
let mut different = Vec::new();

// 1. 联系方式匹配 (最高 60% 权重)
total_weight += 0.6;
let mut contact_matches = 0;
let mut contact_conflicts = 0;

for input_contact in input_contacts {
    for existing_contact in existing_contacts {
        if input_contact.r#type == existing_contact.contact_type &&
            input_contact.value == existing_contact.contact_value {
            contact_matches += 1;
        }
    }
}

if contact_matches >= 2 {
    score += 0.6; // 两个以上联系方式相同, 高置信度
    matched.push(format!("{}", 个联系方式匹配", contact_matches));
} else if contact_matches == 1 {
    // 仅一个联系方式相同, 需要判断是否共用
    let is_shared = check_if_contact_is_shared(&input_contacts[0].value).await?;
    if is_shared {
        score += 0.3; // 共用电话, 降低置信度
        matched.push("1 个联系方式匹配 (可能共用)".to_string());
    } else {
        score += 0.4; // 非共用电话
        matched.push("1 个联系方式匹配".to_string());
    }
} else {
    different.push("无匹配联系方式".to_string());
}

// 2. 姓名 + 其他字段 (40% 权重)
// ... (与之前相同)

let final_score = if total_weight > 0.0 {
    score / total_weight
} else {
    0.5
};

(final_score, matched, different)
}

```

```

/// 检查电话是否被多人共用
async fn check_if_contact_is_shared(contact_value: &str) -> bool {
    let count = sqlx::query_scalar::<_, i64>(
        r#"
        SELECT COUNT(DISTINCT person_id)
        FROM contact_methods
        WHERE contact_value = ? AND invalid_at IS NULL
        "#,
    )
    .bind(contact_value)
    .fetch_one(&pool)
    .await
    .unwrap_or(0);

    count > 1
}

```

6.2 规则总结

匹配情况	置信度	处理方式
2 个以上联系方式相同	95%	自动合并
1 个电话 + 姓名 + 生日相同	90%	建议合并
1 个电话（非共用）+ 姓名相同	70%	建议合并，用户确认
1 个电话（共用）+ 姓名相同	50%	人工审核
仅姓名相同	50%	待确认
姓名 + 生日不同	30%	不同人

七、总结

7.1 核心设计

问题	解决方案
一人多电话	<code>contact_methods</code> 表，一对多关联
多人一电话	电话类型标记 + 共用检测 + 置信度降低
电话变更	<code>invalid_at</code> 字段标记失效，保留历史
主号码识别	<code>is_primary</code> 字段 + <code>persons.primary_phone</code> 冗余
查重误判	多联系方式综合判断，不单靠一个电话

7.2 联系方式类型

类型	代码	图标
手机	mobile	
座机	landline	
工作电话	work_phone	
家庭电话	home_phone	
邮箱	email	
微信	wechat	

7.3 数据量估算

数据类型	人均数量	1 万用户
联系方式	2-3 个	2-3 万条
共用电话	<5%	<1500 条
失效电话	<10%	<3000 条

文档版本 : v1.0

最后更新: 2026 年 3 月 14 日

开发负责人: 王舟

邮箱 : main@mails.wedevs.org

电话: 15378391447

开发负责人: 王舟 | 电话: 15378391447 | 邮箱: main@mails.wedevs.org